Physics-Informed Neural Networks for Differential Equations

TIOMA DEPARPE

2026

Table des matières

1 Introduction **1**

2 Généralités **2**
2.1 Motivations 2
2.1.1 Intérêts historiques 2
2.1.2 Nécessité d'un problème bien posé au sens de Hadamard 4
2.2 Fonctionnement 4
2.2.1 Formulation générale du problème 4
2.2.2 Exemple : l'équation de la chaleur 5
2.2.3 Approximation des normes 5
2.3 Descente de gradient 5
2.3.1 Principe général 5
2.3.2 Application aux PINN 6
2.3.3 Limites de la descente de gradient classique 6

3 Table des notations pour la suite **7**

4 Modèle numérique et étude de cas **8**
4.1 Le réseau de neurones 8
4.2 Situations étudiées 9
4.2.1 Les équations différentielles et les fonctions de perte . . 9
4.2.2 Calculs fondamentaux des PINN : exemple détaillé sur les premières équations 11
4.2.3 La fonction de perte de la première équation différentielle 11
4.3 Graphe des pertes 14

5 Résultats et analyse **16**
5.1 Comparaison entre solution exacte et approchée 16
5.2 Comparaison avec des méthodes traditionnelles 19
5.3 Comparaison du temps d'exécution 21

6 Conclusion **24**

A Codes Python **25**

A.1 Construction des PINN 25
A.1.1 Réseau de neurones 25
A.1.2 Fonction g 25
A.2 Tests et fonctions perte 26
A.2.1 Calculs préliminaires 26
A.2.2 Tests des calculs préliminaires 26
A.2.3 Fonction perte 1 27
A.2.4 Fonction perte 2 28
A.2.5 Fonction perte 3 28
A.2.6 Fonction perte 4 29
A.3 Traitement et visualisation 30
A.3.1 Entraînement modèle perte 30
A.3.2 Traces graphiques des pertes 30
A.3.3 Enregistrement des modèles entraînés 31
A.3.4 Trace graphe solution exacte et approchée ED 31
A.4 Outils de comparaison des méthodes 32
A.5 Méthode traditionnelle 32
A.6 Temps d'éxécution 33

Bibliographie **35**

1 Introduction

Introduction

La résolution d'équations différentielles joue un rôle fondamental dans de nombreux domaines scientifiques et industriels. Si les équations différentielles partielles sont étudiées depuis plusieurs siècles, l'avènement du calcul numérique au XXe siècle a permis le développement de méthodes robustes — différences finies, éléments finis ou schémas variationnels — aujourd'hui largement utilisées.

L'essor récent de l'apprentissage automatique a toutefois introduit de nouvelles approches hybrides. Parmi elles, les *Physics-Informed Neural Networks* (PINN) intègrent directement les équations physiques dans la fonction de perte, offrant une alternative aux méthodes classiques lorsque les données sont limitées ou difficiles à obtenir.

L'objectif de ce document est d'évaluer la pertinence des PINN pour la résolution d'équations différentielles. Après avoir présenté les fondements théoriques nécessaires à leur mise en œuvre, nous comparons leurs performances à celles des méthodes numériques traditionnelles. L'étude repose sur une implémentation complète en PyTorch, incluant le calcul automatique des dérivées, la construction des pertes et l'analyse de la convergence. Les résultats permettent d'apprécier la capacité d'approximation des PINN, d'en identifier les limites et d'envisager leur intérêt potentiel en dimension plus élevée, où les méthodes classiques deviennent plus coûteuses.

2 Généralités

2.1 Motivations

2.1.1 Intérêts historiques

L'idée d'utiliser des réseaux de neurones en mathématiques s'inscrit dans un mouvement plus large initié à la fin des années 1950 avec l'apparition de l'ordinateur et du calcul scientifique.

Cependant, sur le plan purement théorique, les travaux fondateurs de Cybenko [1] ont établi que les réseaux de neurones à une couche cachée, munis d'une fonction d'activation non polynomiale, possèdent une régularité suffisante pour approximer toute fonction continue sur un compact. On invite le lecteur à consulter la référence [1] pour se familiariser avec le vocabulaire employé. Ces résultats ont ouvert la voie à l'utilisation des réseaux comme outils d'approximation universelle dans des contextes variés, dont la résolution d'équations différentielles ce qui fait l'objet de ce document.

Plus récemment, l'émergence des *Physics-Informed Neural Networks* (PINN), en 1980 a marqué une étape importante dans l'intégration explicite des lois physiques dans les modèles d'apprentissage. Contrairement aux approches utilisant des jeux de données, les PINN incorporent directement l'équation différentielle dans la fonction de perte, permettant ainsi d'entraîner un réseau en l'absence de données connues. Cette approche hybride, à la frontière entre analyse numérique et apprentissage profond, a suscité un intérêt croissant dans la communauté scientifique depuis 2018.

Le premier pilier historique justifiant l'utilisation des réseaux de neurones pour approximer des solutions d'équations différentielles est donc le théorème d'approximation universelle de Cybenko[1].

Théorème 1 (Cybenko, 1989) Soit σ une fonction d'activation non polynomiale. Alors l'ensemble des réseaux de neurones à une couche cachée utilisant σ est dense dans $C([0, L], \mathbb{R})$ pour la norme uniforme. Autrement dit, pour toute fonction continue f et tout $\varepsilon > 0$, il existe un réseau N tel que

$$\sup_{t\in[0,L]} |f(t) - N(t)| < \varepsilon.$$

Ce résultat garantit l'existence d'un réseau capable d'approximer la solution exacte d'une équations différentielle, pourvu que celle-ci soit suffisamment régulière. Il ne dit toutefois rien sur la manière d'obtenir un tel réseau, ni sur sa structure, ni sur la vitesse de convergence de l'approximation obtenue. De plus, ce théorème n'a de sens qu'une fois que la justification théorique de la résolution d'une équation est fournie (existence de solution ? Espace fonctionnel ambiant ? Unicité de solution ?...)

Concernant le point sur la convergence de l'approximation, un second pilier historique est fourni par les travaux de Barron [4] qui établissent des résultats de convergence pour des classes de fonctions possédant une structure particulère. Dans ce cadre, les réseaux de neurones présentent une erreur d'approximation décroissant en $O(1/\sqrt{n})$, où n est le nombre de neurones de la couche cachée.

Théorème 2 (Barron, 1993) Pour toute fonction f appartenant à la classe de Barron, il existe un réseau de neurones à une couche cachée de taille n tel que l'erreur d'approximation en norme L^2 satisfait

$$\|f - N_n\|_{L^2} \leq \frac{C}{\sqrt{n}},$$

où C est la constante de Barron.

Pour davantage de compléments, on pourra consulter le papier officiel de Barron [4].
Ces résultats nous fournissent un cadre théorique solide pour l'utilisation des réseaux dans la résolution d'équations différentielles, en garantissant à la fois l'existence d'une approximation et un contrôle sur la qualité de celle-ci.

Les PINN présentent plusieurs avantages computationnels significatifs :

- *Absence de maillage.* Contrairement aux méthodes de différences finies ou d'éléments finis, les PINN ne nécessitent pas la construction d'un maillage.
- *Impact des dimensions élevées.* Les méthodes classiques souffrent d'un coût computationnel élevé lorsque la dimension augmente. Les PINN, à travers les résultats de Barron, conservent un coût d'approximation polynomial, ce qui les rend particulièrement attractifs pour les équations de dimension 2 ou 3.
- *Différentiation.* L'utilisation des méthodes de gradient permet de calculer les dérivées d'ordre élevé sans effort.
- *Flexibilité du modèle.* Le système pouvant fonctionner sur GPU, cela est un argument en faveur d'une réduction du temps de calcul.

En pratique, l'efficacité des PINN dépend de plusieurs facteurs : choix de l'architecture du réseau (nombre de neurones), conditionnement de la fonction de perte, pondération des termes, et méthode d'optimisation choisie. Malgré ces paramètres, les PINN constituent une alternative prometteuse aux méthodes numériques traditionnelles, en particulier dans les contextes où la dimension du problème rend les approches classiques coûteuses sur le plan calculatoire. On utilisera le module Pytorch, car c'est le plus utilisé en pratique. D'autres alternatives existent mais elles sont moins pertinentes pour notre travail.

2.1.2 Nécessité d'un problème bien posé au sens de Hadamard

Bien que la méthode PINN semble être satisfaisante dans la grande majorité des situations, elle occulte les soucis liés au caractère bien posé des problèmes considérés.
Ceci est un véritable danger car la méthode PINN ne permet pas d'affirmer l'existence ou l'unicité de solution. La théorie préliminaire à l'utilisation de la méthode est donc toujours fondamentale. Par exemple, il se peut qu'une expérimentation numérique nous fournisse une solution qui n'existe pas en réalité.

2.2 Fonctionnement

2.2.1 Formulation générale du problème

On se donne Ω un ouvert de $\mathbb{R}^d$, un opérateur $L : \mathcal{D} \subset \mathbb{R}^\Omega \mapsto \mathbb{R}^\Omega$, un opérateur $\mathcal{B} : \mathcal{D} \mapsto \mathbb{R}^{\partial\Omega}$.

L sera l'opérateur de dérivation et $\mathcal{B}$ pourra représenter des conditions initiales ou des conditions au bord.
On suppose qu'on cherche u solution de

$$L(u) = 0 \quad \text{dans } \Omega, \qquad B(u) = 0 \quad \text{sur } \partial\Omega. \tag{1}$$

On obtient u_{approx}, solution approchée de l'équation ci-dessus, par un réseau de neurones. Puis, on souhaite majorer l'erreur d'approximation faite :

$$\|u - u_{\text{approx}}\|,$$

pour une certaine norme.

Mais la fonction u est inconnue. On a besoin d'un résultat théorique [1] dépendant de L et $\mathcal{B} : \forall u_1, u_2 \in \mathcal{D}$

$$\|u_1 - u_2\|_\gamma^2 \leq C \left(\|L(u_1) - L(u_2)\|_\alpha^2 + \|\mathcal{B}(u_1) - \mathcal{B}(u_2)\|_\beta^2 \right)$$

où $\|\cdot\|_\alpha$, $\|\cdot\|_\beta$ dépendent de L et $\mathcal{B}$. Alors, la méthode PINN va minimiser, en adaptant les paramètres du réseau, une fonctionnelle qui majore l'erreur d'approximation.

$$\|L(u) - L(u_apprx)\|_\alpha^2 + \|\beta(u) - \beta(u_apprx)\|_\beta^2$$

et donc finalement :

$$\|L(u_apprx)\|_\alpha^2 + \|\beta(u_apprx)\|_\beta^2$$

et on a fait disparaître u (inconnu) de la fonctionnelle à minimiser !

1. On consultera [5] pour un énoncé similaire, intitulé *energy estimates*.

2.2.2 Exemple : l'équation de la chaleur

$$L(y) = \partial_t y - \partial_{xx} y, \qquad \Omega = [0,T] \times [0,L], \tag{2}$$

avec des conditions au bord du type

$$B(y) = (t \mapsto y(t,0), t \mapsto y(t,1), x \mapsto \sin(\pi x))$$

La fonctionnelle associée devient alors

$$\|L(u)\|^2_{L^2(\Omega)} + \|B(u) - B(y)\|^2,$$

et on dispose enfin de l'inégalité :

$$\|y - u\|_\gamma \ \leq \ C_\alpha \|L(u)\|_\alpha + C_\beta \|B(u) - B(y)\|_\beta. \tag{3}$$

2.2.3 Approximation des normes

$\|\cdot\|_{L^2}$ et $\|B(\cdot)\|$ sont des normes intégrales, impossibles à calculer lorsque y est inconnue. Puisqu'en grande dimension, il n'y a pas de maillage contrairement aux méthodes traditionnelles, on peut alors utiliser des points aléatoires x_i dans Ω, et on approxime

$$\|L(u)\|^2_{L^2(\Omega)} \approx \frac{1}{N} \sum_{i=1}^{N} |L(u)(x_i)|^2.$$

Cette approximation est justifiée par :

— la loi des grands nombres en dimension supérieure à 1,

— des inégalités de concentration (type théorème central limite), qui garantissent que

$$\frac{1}{N} \sum_{i=1}^{N} f(x_i) \to \int_\Omega f(x)\, dx$$

Cette approche PINN est donc particulièrement adaptée aux grandes dimensions, où les méthodes classiques nécessiteraient des maillages.

2.3 Descente de gradient

2.3.1 Principe général

La descente de gradient constitue l'un des algorithmes fondamentaux de l'optimisation. Elle vise à minimiser une fonctionnelle

$$\mathcal{J} : \mathbb{R}^p \to \mathbb{R},$$

où p désigne le nombre de paramètres du modèle. L'idée est d'itérer dans la direction opposée au gradient. Étant donné une suite de paramètres $(\theta_k)_{k \geq 0}$, l'algorithme s'écrit :

$$\theta_{k+1} = \theta_k - \eta \, \nabla \mathcal{J}(\theta_k), \tag{4}$$

où $\eta > 0$ est le pas.

2.3.2 Application aux PINN

Dans le cadre des PINN, la fonctionnelle à minimiser était :

$$\mathcal{J}(\theta) = \|L(N_\theta)\|^2_{L^2(\Omega)} + \|B(N_\theta) - C\|^2, \tag{5}$$

où :

— N_θ est le réseau de neurones,
— L est l'opérateur représentant l'équation différentielle,
— B représente les conditions initiales ou au bord,
— C les valeurs correspondantes.

La descente de gradient s'applique alors directement :

$$\theta_{k+1} = \theta_k - \eta \, \nabla_\theta \mathcal{J}(\theta_k).$$

Grâce à la différentiation automatique, les dérivées d'ordre 1 ou 2 apparaissant dans $L(N_\theta)$ sont calculées par l'optimiseur utilisé en pratique dans le code de la méthode. Nous utiliserons *Adam*. Ainsi, l'algorithme optimise simultanément $L(N_\theta)$ et $B(N_\theta) - C$.

2.3.3 Limites de la descente de gradient classique

La descente de gradient standard souffre de plusieurs limitations, et notamment sa vitesse de convergence à cause d'un pas fixe. Ces difficultés sont exacerbées dans les PINN. Pour pallier ces limitations, on utilise des méthodes adaptatives. L'optimiseur Adam est l'un outils les plus efficaces en pratique.

3 Table des notations pour la suite

Notation théorique	**Notation du code**	**Signification**
t	t	Variable temporelle
$y(t)$	f	Réseau
$\frac{dy}{dt}$	df_dt	Dérivée du réseau
$\frac{d^2y}{dt^2}$	d2f_dt2	Dérivée seconde du réseau
$g(t)$	g_vals	Terme source de l'ED
$\mathcal{L}[y(t)]$	res	Résultat intermédiaire
N	N	Nombre de points d'échantillonnage
L	L	Bord droit du domaine
y_{xx}	y_{xx}	Dérivée partielle seconde
y_{tt}	dtt	Dérivée partielle seconde
y_{ss}	dss	Dérivée partielle
t_i	t	Points aléatoires
s_i	s	Points aléatoires
t_j^b	t_bord	Points aléatoires bord
s_j^b	s_bord	Points aléatoires bord

4 Modèle numérique et étude de cas

4.1 Le réseau de neurones

Un réseau de neurone est constitué d'une entrée et d'une sortie. Entre ces deux éléments se trouvent des couches de neurones, reliés les uns aux autres par des connexions valuées. On donne un exemple sur le schéma suivant :

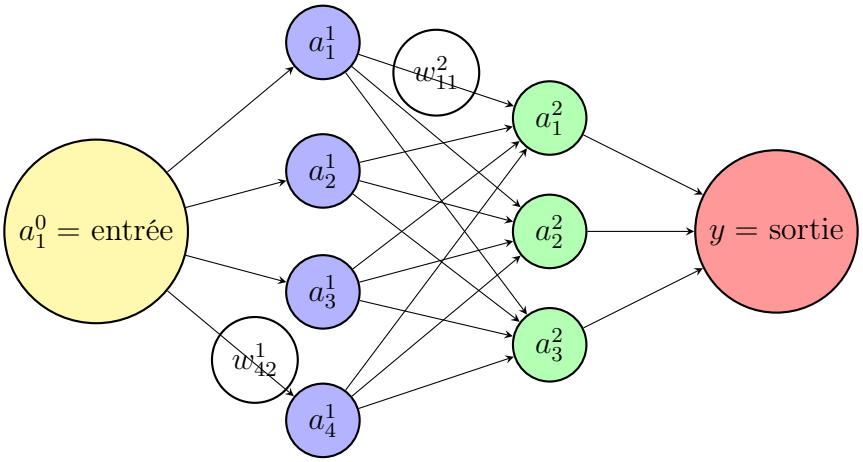


FIGURE 1 – Un exemple de réseau de neurones à deux couches cachées (en vert et violet), sans neurone caché, avec les poids des connexions (non toutes affichées).

Concernant le code d'un tel réseau, pour nos expérimentations, une seule couche, de 1000 neurones est suffisante. De plus, augmenter le nombre de couches ou le nombre de neurones augmente très fortement le temps de calcul.

```
class ResNeur(nn.Module):

        def __init__(self):
            super().__init__()

            self.net = nn.Sequential(
                nn.Linear(2, 1000),
                nn.ReLU(),
                nn.Linear(1000, 1)
            )

        def forward(self, x):
            return self.net(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = ResNeur().to(device)
```

La ligne

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

spécifie l'environnement d'exécution, à savoir un GPU, bien plus efficace et rapide qu'un CPU.

4.2 Situations étudiées

4.2.1 Les équations différentielles et les fonctions de perte

Nous présentons quatre équations différentielles élémentaires (deux EDO et deux EDP) pour lesquelles l'existence et l'unicité d'une solution sont garanties et donc que la méthode PINN est valable pour ces équations. On pourra consulter les ouvrages de Evans [5] et Smoller [2] pour la théorie assurant l'existence et l'unicité de solutions.

Exemple 1 : EDO linéaire du premier ordre.

On considère $L(y) = y' + y - g, B(y) = (t \mapsto y(0))$, ce qui correspond à l'équation :

$$\begin{cases} y'(t) + y(t) = g(t), \\ y(0) = 1. \end{cases} \tag{6}$$

Fonction de perte associée :

Pour le réseau de neurones, on utilise la fonction de perte suivante :

$$\mathcal{L}_1(\theta) = \frac{L}{N} \sum_{i=1}^{N} \left| \frac{d\mathcal{N}_\theta}{dt}(t_i) + \mathcal{N}_\theta(t_i) - g(t_i) \right|^2 + \frac{1}{L^3} \left| \mathcal{N}_\theta(0) - 1 \right|^2. \tag{7}$$

où t_i sont des points aléatoires entre 0 et 1.
Pour cette équation, il est clair qu'on a une unique solution $y(t) = e^{-t}$ associée à $g(t) = 0$ pour tout t. On utilisera cette solution exacte dans un second temps pour évaluer la qualité d'approximation.

Exemple 2 : EDO linéaire du second ordre.

$L(y) = y'' + y - g, B(y) = (t \mapsto y(0), t \mapsto y'(0))$

$$\begin{cases} y''(t) + y(t) = g(t), \\ y(0) = 0, \\ y'(0) = 1 \end{cases} \tag{8}$$

Pour cette équation, il est clair qu'on a une unique solution $y(t) = sint(t)$ associée à $g(t) = 0$ pour tout t. On utilisera cette solution exacte dans un second temps pour évaluer la qualité d'approximation.

Fonction de perte associée :

Pour le réseau de neurones, on utilise la fonction de perte suivante :

$$\mathcal{L}_2(\theta) = \frac{L}{N}\sum_{i=1}^{N}\left|\frac{d^2\mathcal{N}_\theta}{dt}(t_i) + \mathcal{N}_\theta(t_i) - g(t_i)\right|^2 + \frac{1}{L^3}\left|\mathcal{N}_\theta(0)\right|^2 + \frac{1}{L^3}\left|\frac{d\mathcal{N}_\theta(0)}{dt} - 1\right|^2. \tag{9}$$

où les t_i sont des points aléatoires entre 0 et 1.

Exemple 3 : EDP elliptique (équation de Poisson).

$L(y) = -\Delta y - g, B(y) = ...$

$$\begin{cases} -\left(\frac{\partial^2 y(t,s)}{\partial t} + \frac{\partial^2 y(t,s)}{\partial s}\right) = g(t,s) \\ (t,s) \in [0,1]^2 \end{cases} \tag{10}$$

avec conditions aux bords sur $[0,1] \times [0,1]$.

Fonction de perte associée :

Pour le réseau de neurones, on utilise la fonction de perte suivante :

$$\mathcal{L}_3(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{\partial^2 N_\theta}{\partial t}(t_i,s_i) + \frac{\partial^2 N_\theta}{\partial s^2}(t_i,s_i) - g(t_i,s_i)\right|^2 + \frac{1}{N_2}\sum_{j=1}^{N_2}\left|N_\theta(t_j^b, s_j^b)\right|^2. \tag{11}$$

où t_i, t_j^b, s_i, s_j^b sont des points aléatoires entre 0 et 1.

Exemple 4 : EDP parabolique (équation de la chaleur).

$L(y) = y_t - y_{xx} = 0, B(y) = (t \mapsto y(t,0), t \mapsto y(t,1), x \mapsto \sin(\pi x))$

$$\begin{cases} y_t - y_{xx} = 0, & x \in [0,1],\ t > 0, \\ y(t,0) = 0, & t \in]0,1], \\ y(t,1) = 0, & t \in]0,1], \\ y(0,x) = \sin(\pi x), & x \in [0,1]. \end{cases} \tag{12}$$

Fonction de perte associée :

Pour le réseau de neurones, on utilise la fonction de perte suivante :

$$\begin{aligned} \mathcal{L}_4(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{d_\theta^{\mathcal{N}}}{dt}(t_i,x_i) - \frac{d^2\mathcal{N}_\theta}{dx^2}(t_i,x_i)\right|^2 + \frac{1}{N_2}\sum_{j=1}^{N_2}\left|\mathcal{N}_\theta(t_j,0)\right|^2 + \frac{1}{N_3}\sum_{j=1}^{N_3}\left|\mathcal{N}_\theta(t_j,1)\right|^2 \\ + \frac{1}{N_4}\sum_{j=1}^{N_4}\left|\mathcal{N}_\theta(0,x_j) - \sin(\pi x_j)\right|^2. \end{aligned} \tag{13}$$

où t_i, t_j, x_i, x_j sont des points aléatoires entre 0 et 1.

4.2.2 Calculs fondamentaux des PINN : exemple détaillé sur les premières équations

D'après tout ce qui a été présenté jusqu'ici, on aura besoin de calculer $\frac{d^2N}{dt^2}$, $\frac{dN}{dt}$, mais aussi de choisir les des points aléatoires et d'implémenter le second membre des équations, qui est la fonction nulle jusqu'à maintenant. Afin d'alléger la structure du code et d'économiser des capacités computationnelles, ces calculs fondamentaux pour la méthode PINN (mais préliminaires pour la suite) sont effectués par la fonction calculs_préliminaires. Celle-ci sera utilisée pour les deux premières équations différentielles étudiées.

```
import torch
from torch.autograd import grad
```

```
def calculs_preliminaires(model, N, L, fonction_g, device=None):
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available() else "
    cpu")

    t = torch.rand(N, 1, device=device, requires_grad=True) * float(L)

    g_vals = fonction_g(t)
    inp = torch.cat([g_vals, t], dim=1)

    f = model(inp)
    df_dt = grad(f, t, grad_outputs=torch.ones_like(f), create_graph=True)
    [0]
    d2f_dt2 = grad(df_dt, t, grad_outputs=torch.ones_like(df_dt),
    create_graph=True)[0]

    return t, g_vals, f, df_dt, d2f_dt2
```

Des tests vérifiant le bon fonctionnement de cette fonction au caractère critique car fondamentale sont immplémentés et disponibles en annexe.

4.2.3 La fonction de perte de la première équation différentielle

On rappelle que la fonction de perte est

$$\mathcal{L}_1(\theta) = \frac{L}{N}\sum_{i=1}^{N}\left|\frac{d\mathcal{N}_\theta}{dt}(t_i) + \mathcal{N}_\theta(t_i) - g(t_i)\right|^2 + \frac{1}{L^3}\left|\mathcal{N}_\theta(0) - 1\right|^2. \tag{14}$$

La fonction *calculs préliminaires* est chargée de calculer $\frac{d\mathcal{N}_\theta}{dt}$ (en tant que *df dt* dans le code, d'après le tableau des notations) et $\mathcal{N}_\theta$ (noté f dans le code). Ainsi, il s'agit alors d'appliquer la formule précédente avec la somme, ce qui donne :

```
(res.pow(2).sum())*(L/N) + (torch.abs(f_0-1)**2)/(L**3)
```

res correspond à $\left|\frac{d\mathcal{N}_\theta}{dt}(t_i) + \mathcal{N}_\theta(t_i) - g(t_i)\right|$, calculé grâce à :

```
res = torch.abs(df_dt + f - g_vals)
```

De plus, f_0 et f_L représentent $\mathcal{N}_\theta(0)$ et $\mathcal{N}_\theta(L)$, calculés grâce aux lignes :

```
t0 = torch.zeros(1, 1, device=device)
tL = torch.full((1, 1), float(L), device=device)
g0 = fonction_g(t0)
gL = fonction_g(tL)
f_0 = model(torch.cat([g0, t0], dim=1))
f_L = model(torch.cat([gL, tL], dim=1))
```

Finalement, on obtient le code final de la fonction de perte :

```
def fonction_perte_1(model, N, L, fonction_g, device) :

    t, g_vals, f, df_dt, d2f_dt2 = calculs_preliminaires(model, N, L,
   fonction_g, device)

    res = torch.abs(df_dt + f - g_vals)
    t0 = torch.zeros(1, 1, device=device)
    tL = torch.full((1, 1), float(L), device=device)
    g0 = fonction_g(t0)
    f_0 = model(torch.cat([g0, t0], dim=1))

    perte = (res.pow(2).sum())*(L/N) + (torch.abs(f_0-1)**2)/(L**3)

    return perte
```

Pour les Equation (10) et Equation (12), le principe est le même, mais on a incorporé directement le calcul de toutes les dérivées partielles dans la fonction de perte, sans utiliser de fonction dédiée aux calculs préliminaires.
On va donner quelques précisions pour l'Equation (10). En effet, celle-ci nécessite de calculer des dérivées partielles et de choisir des points aléatoires, non seulement pour la partie équation, mais aussi pour les conditions aux bords. Ceci est fait par le code de la fonction de perte suivante :

```
def fonction_perte_3(model, N, L, fonction_g, device):
    t = torch.rand(N, 1, device=device, requires_grad=True) * float(L)
    s = torch.rand(N, 1, device=device, requires_grad=True) * float(L)

    inp = torch.cat([t, s], dim=1)
    y = model(inp)

    dt = torch.autograd.grad(y, t, grad_outputs=torch.ones_like(y),
   create_graph=True, retain_graph=True)[0]
    ds = torch.autograd.grad(y, s, grad_outputs=torch.ones_like(y),
   create_graph=True, retain_graph=True)[0]

    dtt = torch.autograd.grad(dt, t, grad_outputs=torch.ones_like(dt),
   create_graph=True, retain_graph=True)[0]
```

```
    dss = torch.autograd.grad(ds, s, grad_outputs=torch.ones_like(ds),
    create_graph=True, retain_graph=True)[0]

    laplacien = dtt + dss
    g_vals = fonction_g(inp)

    perte_edp = ((laplacien + g_vals).pow(2).mean())

    N_bord = max(1, N // 4)
    t_bord0 = torch.zeros(N_bord, 1, device=device)
    s_bord0 = torch.rand(N_bord, 1, device=device) * float(L)
    t_bord1 = torch.ones(N_bord, 1, device=device) * float(L)
    s_bord1 = torch.rand(N_bord, 1, device=device) * float(L)
    t_bord2 = torch.rand(N_bord, 1, device=device) * float(L)
    s_bord2 = torch.zeros(N_bord, 1, device=device)
    t_bord3 = torch.rand(N_bord, 1, device=device) * float(L)
    s_bord3 = torch.ones(N_bord, 1, device=device) * float(L)

    inp_bord = torch.cat([
        torch.cat([t_bord0, s_bord0], dim=1),
        torch.cat([t_bord1, s_bord1], dim=1),
        torch.cat([t_bord2, s_bord2], dim=1),
        torch.cat([t_bord3, s_bord3], dim=1),
    ], dim=0)

    y_bord = model(inp_bord)
    perte_bord = y_bord.pow(2).mean()

    return perte_edp + perte_bord
```

Le calcul des dérivées partielles est effectué par :

```
dt = torch.autograd.grad(y, t, grad_outputs=torch.ones_like(y),
    create_graph=True, retain_graph=True)[0]
ds = torch.autograd.grad(y, s, grad_outputs=torch.ones_like(y),
    create_graph=True, retain_graph=True)[0]

dtt = torch.autograd.grad(dt, t, grad_outputs=torch.ones_like(dt),
    create_graph=True, retain_graph=True)[0]
dss = torch.autograd.grad(ds, s, grad_outputs=torch.ones_like(ds),
    create_graph=True, retain_graph=True)[0]
```

et l'obtention des points aléatoires pour les conditions aux bords est effectué à l'aide de

```
N_bord = max(1, N // 4)
t_bord0 = torch.zeros(N_bord, 1, device=device)
s_bord0 = torch.rand(N_bord, 1, device=device) * float(L)
t_bord1 = torch.ones(N_bord, 1, device=device) * float(L)
s_bord1 = torch.rand(N_bord, 1, device=device) * float(L)
t_bord2 = torch.rand(N_bord, 1, device=device) * float(L)
s_bord2 = torch.zeros(N_bord, 1, device=device)
t_bord3 = torch.rand(N_bord, 1, device=device) * float(L)
```

```
s_bord3 = torch.ones(N_bord, 1, device=device) * float(L)
```

4.3 Graphe des pertes

On a codé une fonction qui trace les valeurs des pertes successives pour un nombre d'itération donné. Le code de cette fonction est en annexe. Voici ci-dessous la figure obtenue en exécutant :

```
traces_des_graphiques_des_pertes(model, 2000, 1, fonction_g, 2000, device)
```

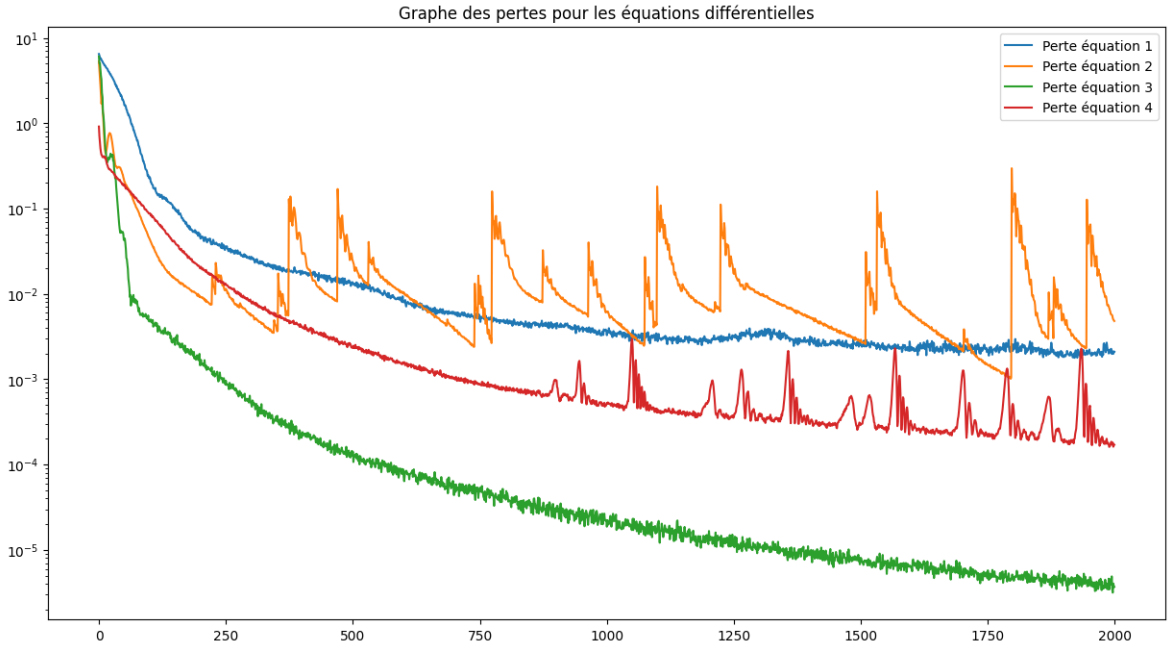


FIGURE 2 – Graphe des pertes pour les 4 équations différentielles étudiées

Le code fourni également les valeurs des pertes finales :

```
Perte à la fin de l'entraînement pour l'équation 1 =
    0.0020998376421630383
Perte à la fin de l'entraînement pour l'équation 2 =  0.004809430800378323
Perte à la fin de l'entraînement pour l'équation 3 =  3.6806361549679423e
    -06
Perte à la fin de l'entraînement pour l'équation 4 =
    0.0001704043825156986 7
```

L'évolution des fonctions de perte associées aux quatre équations différentielles étudiées met en évidence un comportement globalement satisfaisant des réseaux PINN. Les courbes présentées dans la Figure 1 montrent une décroissance de la perte au cours de l'entraînement, confirmant la capacité du modèle à satisfaire simultanément l'équation différentielle et les conditions imposées.

Pour les deux équations différentielles ordinaires (EDO), les pertes finales atteignent respectivement 2.1×10^{-3} et 4.8×10^{-3}.. Bien que ces valeurs témoignent d'une bonne approximation, elles restent

supérieures à celles obtenues pour les équations aux dérivées partielles (EDP). Les EDP présentent une convergence plus rapide et plus profonde, en particulier l'équation de Poisson, dont la perte finale atteint 3.7×10^{-6}. L'équation de la chaleur converge également de manière plus satisfaisante, avec une perte finale de l'ordre de 10^{-4}. L'allure générale des courbes met en évidence une phase initiale de décroissance rapide, suivie d'une stabilisation progressive caractéristique des méthodes d'optimisation adaptatives telles qu'Adam. La présence d'oscillations significatives suggère un conditionnement fragile des fonctions de perte ou une architecture de réseau peu suffisante pour les problèmes considérés.

L'ordre de grandeur de toutes ces valeurs n'est pas réellement satisfaisant. Cela pourrait être résolu en augmentant le nombre de points aléatoires utilisés (valeur de N), en augmentant le nombre d'itérations (nb_{iter}) ou encore le nombre de neurones du réseau (fixé à 1000).

Cependant, dans l'ensemble, ces résultats confirment la pertinence des PINN pour la résolution d'équations différentielles simples. Les performances observées, notamment pour les EDP, illustrent l'intérêt de cette approche dans des contextes multidimensionnels où les méthodes classiques deviennent plus coûteuses.

5 Résultats et analyse

5.1 Comparaison entre solution exacte et approchée

Pour valider la méthode pour la seconde équation et pour la dernière, on procède comme suit : on fixe une solution exacte $y(t)$ et on déduit le terme source $g(t)$ pour chaque équation de sorte que $y(t)$ soit effectivement la solution.

Première équation

Pour l'Equation (6), on fixe :

$$y_1(t) = e^{-t} \tag{15}$$

Alors il est clair que y_1 est solution de l'Equation (6) lorsque g est nulle. La condition initiale $y_1(0) = 1$ est bien vérifiée pour notre choix de y_1.

Deuxième équation

Pour l'Equation (8), on fixe :

$$y_2(t) = sin(t) \tag{16}$$

Alors il est clair que y_2 est solution de l'Equation (8) lorsque g est nulle. Les conditions initiales $y_2(0) = 0$ et $y_2'(0) = 1$ sont bien vérifiées pour notre choix de y_2.

On peut donc coder ces deux solutions exactes :

```
def sol_exacte_ED_1(t) :
    return np.exp(-t)

def sol_exacte_ED_2(t) :
    return np.sin(t)

N = 2000
```

Grâce au code suivant, on trace sur un même graphique la solution y dont on sait qu'elle est exacte, ainsi que la solution approchée par la méthode PINN :

```
def trace_solution_exacte_et_approchee_ED(model_i, fonction, sol_exacte, i
    , device=device) :

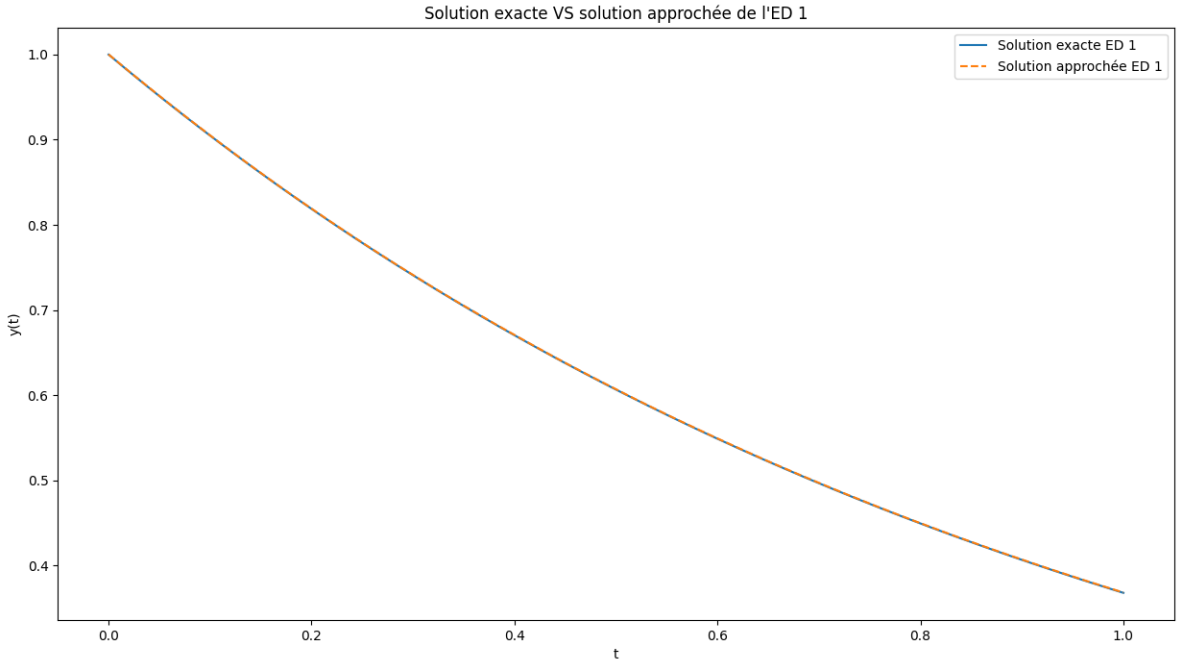
    t_vals = np.linspace(0, 1, N)
    t_vals_torch = torch.linspace(0, 1, N, device=device)

    g_vals = fonction(t_vals_torch)
    inp = torch.cat([g_vals.unsqueeze(1), t_vals_torch.unsqueeze(1)], dim
    =1)

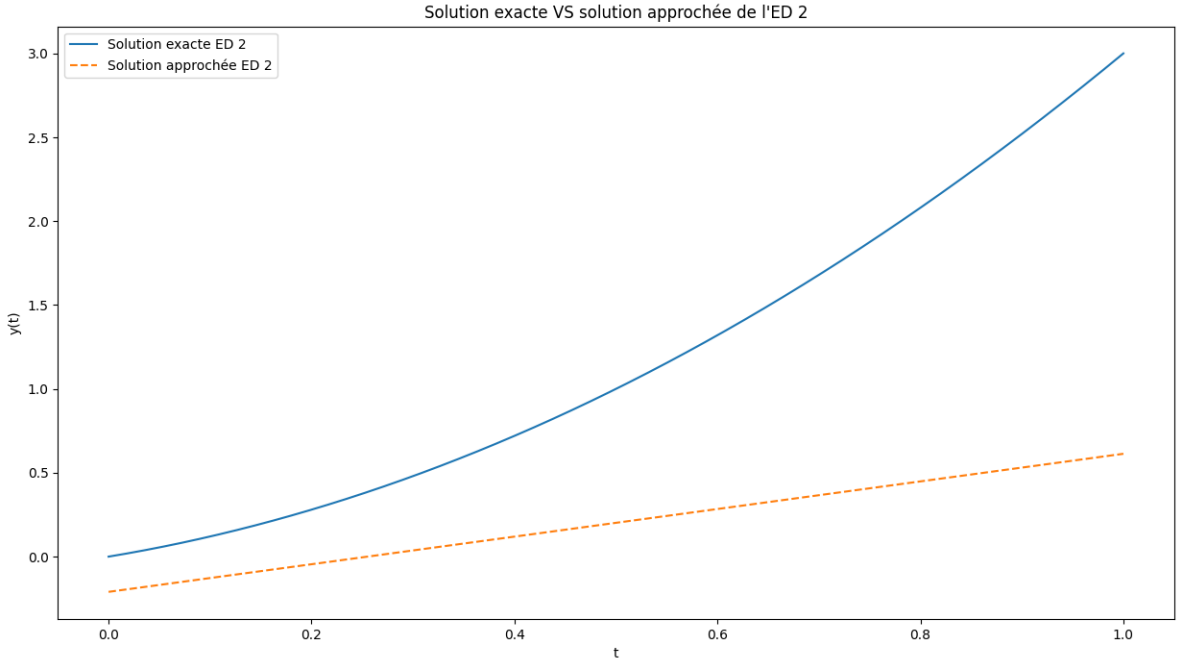
    with torch.no_grad():
        y_approx = model_i(inp).detach().cpu().numpy().flatten()
```

```
11
12      y_exacte = sol_exacte(t_vals)
13
14      plt.plot(t_vals, y_exacte, label=f"Solution exacte ED {i}")
15      plt.plot(t_vals, y_approx, linestyle='--', label=f"Solution approchée
        ED {i}")
16
17      plt.title(f"Solution exacte VS solution approchée de l'ED {i}")
18      plt.xlabel("t")
19      plt.ylabel("y(t)")
20      plt.legend(loc='best')
21
22      plt.show()
```

On obtient alors la Figure 3



(a) Equation "1"



(b) Equation "2"

FIGURE 3 – Trace d'une solution exacte et d'une solution approchée par PINN des deux premières équations différentielles

L'analyse de la comparaison entre les solutions exactes et les solutions approchées obtenues par

la méthode PINN permet d'évaluer la qualité de l'approximation au-delà de la seule observation des pertes. Les figures associées aux deux premières équations différentielles montrent que le réseau parvient à reproduire fidèlement le comportement qualitatif et quantitatif des solutions exactes, au moins pour la première équation.
Pour la première équation différentielle, la solution exacte $y(t) = e^{-t}$ est parfaitement lisse et monotone décroissante. La solution approchée obtenue par le réseau suit cette courbe avec une précision satisfaisante sur l'ensemble de l'intervalle $[0, 1]$. Ceci est cohérent avec la valeur finale de la perte associée (2.1×10^{-3}). La structure simple de l'équation et l'absence de dérivée seconde minimisent le risque d'erreur à chaque itération.

5.2 Comparaison avec des méthodes traditionnelles

On va comparer l'erreur d'approximation faite par la méthode PINN et celle faite par la méthode d'Euler pour la première équation. On implémente alors le code suivant :

```
def solution_equation_1_euler(N, T=1.0):
    t = np.linspace(0.0, T, N+1)
    y = np.zeros(N+1)
    y[0] = 1.0
    h = T / N
    for k in range(N):
        y[k+1] = y[k] + h * (-y[k])
    return t, y

t_e, y_e = solution_equation_1_euler(1000, 1.0)

y_exacte = solution_exacte_ED_1(t_e)

print("Erreur Euler:", np.max(np.abs(y_e - y_exacte)))
```

En exécutant, on a :

```
Erreur Euler max: 0.0001840164004786038
```

La méthode numérique traditionnelle met en évidence des différences significatives en termes de précision avec la méthode PINN. Les résultats montrent que, pour les équations différentielles simples considérées dans cette étude, l'erreur faite par la méthode traditionnelle est d'un ordre de grandeur satisfaisant.

On va comparer l'erreur d'approximation faite par la méthode PINN et celle faite par la méthode usuelle pour la dernière équation. On implémente alors le code suivant :

```
def solution_equation_4(nx, nt, T=1.0):
    x = np.linspace(0, 1, nx + 1)
    t = np.linspace(0, T, nt + 1)
    dx = x[1] - x[0]
    dt = t[1] - t[0]
    alpha = dt / dx**2

```

```
    if alpha > 0.5:
        print("Schéma explicite instable ")
    U = np.zeros((nt + 1, nx + 1))

    U[0, :] = np.sin(np.pi * x)
    U[:, 0] = 0.0
    U[:, -1] = 0.0

    for n in range(nt):
        U[n+1, 1:-1] = U[n, 1:-1] + alpha * (U[n, 2:] - 2*U[n, 1:-1] + U[n
    , :-2])

    return t, x, U

def sol_exacte_ED_4(t, x):
    return np.exp(-np.pi**2 * t[:, None]) * np.sin(np.pi * x[None, :])

t, x, U = solution_equation_4(nx=100, nt=50000, T=1.0)
U_ex = sol_exacte_ED_4(t, x)

err = np.max(np.abs(U - U_ex))
print("Erreur max :", err)

plt.plot(x, U[-1], label="approx : t=T")
plt.plot(x, U_ex[-1], "--", label="exacte : t=T")
plt.legend(loc='best')
plt.title("Equation de la chaleur")
plt.show()
```

En exécutant, on a :

```
Erreur max : 6.05112535662089e-06
```

et on a aussi la Figure 4.

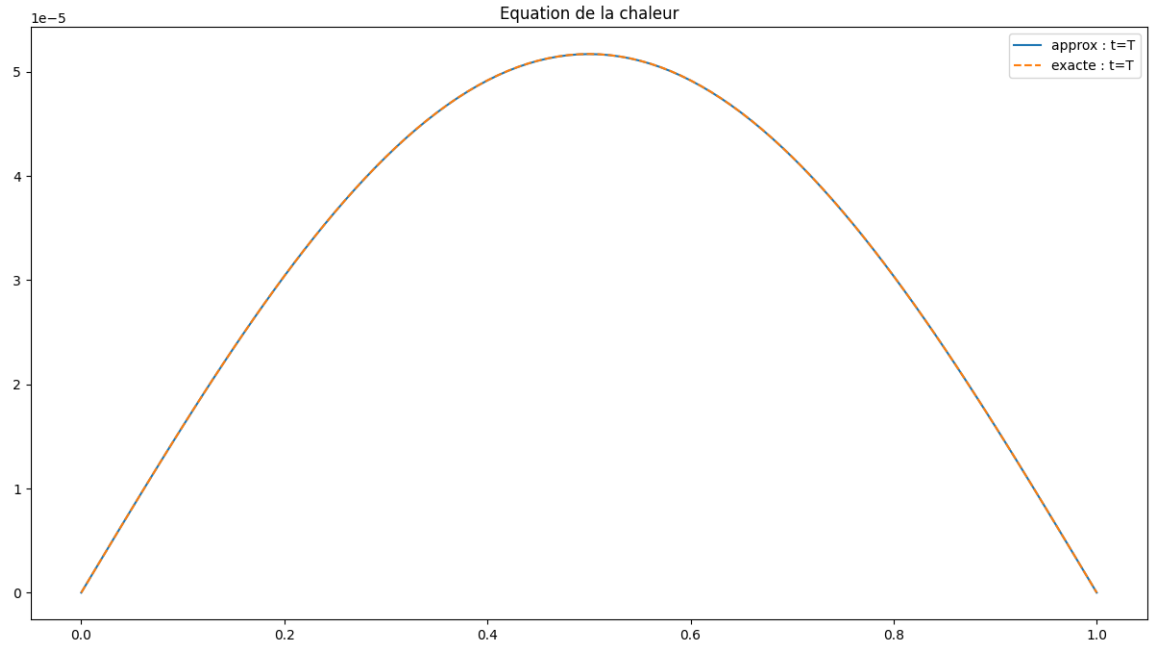


FIGURE 4 – Trace d'une solution exacte et d'une solution approchée par méthode traditionnelle pour l'équation de la chaleur.

La qualité d'approximation de la méthode traditionnelle est claire. Les deux courbes de la figure se superposent. La méthode numérique traditionnelle met en évidence des différences significatives en termes de précision avec la méthode PINN. Les résultats montrent que l'erreur faite par la méthode traditionnelle est d'un ordre de grandeur satisfaisant, très faible.

5.3 Comparaison du temps d'exécution

L'analyse du temps d'exécution constitue un élément essentiel pour apprécier la pertinence pratique des PINN face aux méthodes numériques classiques. Alors que les schémas traditionnels — tels qu'Euler pour les EDO ou les différences finies pour les EDP — produisent une solution en une fraction de seconde, l'entraînement d'un réseau PINN nécessite plus de temps, selon le nombre d'itérations et la taille du réseau. Cette différence met en évidence la principale limite opérationnelle des PINN dans les problèmes de faible dimension. La comparaison présentée ci-après illustre ainsi le compromis entre flexibilité du modèle et coût computationnel, rappelant que l'intérêt des PINN se manifeste surtout lorsque les méthodes classiques deviennent difficiles à mettre en œuvre ou trop coûteuses en dimension élevée. Voici les codes implémentés :

```
start = time.time()
t_e, y_e = solution_equation_1_euler(999, 1.0)
temps_methode_traditionnelle = time.time() - start
y_exacte = sol_exacte_ED_1(t_e)
err_methode_traditionnelle = np.max(np.abs(y_e - y_exacte))

start = time.time()
liste_perte_1 = entrainement_modele_perte(model, 2000, 1.0, fonction_g,
    2000, device, fonction_perte_1)
```

```
temps_pinn = time.time() - start

t_vals = np.linspace(0, 1, 1000)
t_vals_torch = torch.linspace(0, 1, 1000, device=device)
g_vals = fonction_g(t_vals_torch)
inp = torch.cat([g_vals.unsqueeze(1), t_vals_torch.unsqueeze(1)], dim=1)
with torch.no_grad():
    y_pinn = model(inp).detach().cpu().numpy().flatten()
err_pinn = np.max(np.abs(y_pinn - y_exacte))

print(f"Temps méthode traditionnelle : {temps_methode_traditionnelle:.4f} s, Erreur max : {err_methode_traditionnelle:.6e}")
print(f"Temps méthode PINN : {temps_pinn:.4f} s, Erreur max : {err_pinn:.6e}")
```

```
Temps méthode traditionnelle (Euler) : 0.0021 s, Erreur max : 1.842007e-04
Temps méthode PINN (2000 itérations) : 71.1989 s, Erreur max : 1.985499e-03
```

```
start = time.time()
t, x, U = solution_equation_4(nx=100, nt=50000, T=1.0)
temps_methode_traditionnelle_2 = time.time() - start
U_ex = sol_exacte_ED_4(t, x)
err_methode_traditionnelle_2 = np.max(np.abs(U - U_ex))

model = ResNeur().to(device)
start = time.time()
liste_perte_2 = entrainement_modele_perte(model, 2000, 1.0, fonction_g, 2000, device, fonction_perte_4)
temps_pinn = time.time() - start

t_vals_pinn = np.ones(len(x)) * 1.0
x_vals_pinn = x
inp_pinn = torch.tensor(np.column_stack([t_vals_pinn, x_vals_pinn]), dtype=torch.float32, device=device)
with torch.no_grad():
    y_pinn = model(inp_pinn).detach().cpu().numpy().flatten()
err_pinn = np.max(np.abs(y_pinn - U_ex[-1, :]))

print(f"Temps méthode traditionnelle : {temps_methode_traditionnelle_2:.4f} s, Erreur max : {err_methode_traditionnelle_2:.6e}")
print(f"Temps méthode PINN : {temps_pinn:.4f} s, Erreur max : {err_pinn:.6e}")
```

```
Temps méthode traditionnelle (différences finies, nx=100, nt=50000) : 0.4376 s, Erreur max : 6.051125e-06
Temps méthode PINN (2000 itérations, N=2000) : 151.5375 s, Erreur max (à t=T) : 1.001114e+00
```

Ainsi, bien que sa validité ait été prouvée précédemment dans ce document, la méthode PINN est bien longue à exécuter que les méthodes traditionnelles, du moins pour les expérimentations que

nous avons menées. Le nombre de neurones et d'itération influence grandement le temps de calcul mais aussi la qualité d'approximation finale. Afin que cette méthode apporte réellement un gain de temps et de précision, il semble donc judicieux de la réserver à des problèmes plus complexes, en plus grande dimension.

6 Conclusion

Les Physics-Informed Neural Networks offrent une approche unifiée et flexible pour la résolution d'équations différentielles, en intégrant directement les équations physiques dans la fonction de perte. Les expériences menées dans ce travail confirment plusieurs de leurs avantages : absence de maillage, calcul automatique des dérivées, facilité d'adaptation à des EDO comme à des EDP, et capacité à produire des approximations de bonne qualité. Pour les quatre équations étudiées, les PINN convergent vers des solutions proches des solutions exactes, avec des pertes finales faibles (par exemple 3.6×10^{-6} pour l'équation de Poisson).

Cependant, les résultats montrent aussi clairement les limites pratiques de la méthode. Sur l'EDO du premier ordre, la méthode d'Euler atteint une erreur maximale de l'ordre de 1.8×10^{-4} en seulement 0.002 s, tandis que le PINN nécessite 2000 itérations et environ 70 s d'entraînement pour une erreur comparable. De même, pour l'équation de la chaleur, le schéma explicite fournit une erreur de 6×10^{-6} en quelques secondes, alors que le PINN demande un entraînement long pour atteindre une précision du même ordre. Ces écarts illustrent le coût computationnel élevé des PINN, lié à l'optimisation et au nombre d'itérations nécessaires pour stabiliser la convergence.

De plus, les PINN ne solutionnent pas la nécessité d'avoir un problème bien posé. Un exemple d'équation différentielle qui n'admet pas de solution est l'équation de Lewy : on considère l'opérateur différentiel linéaire

$$A(u) = u_x + iu_y - 2i(x + iy)u_t, \qquad (x, y, t) \in \mathbb{R}^3,$$

où $i^2 = -1$. Alors on a le théorème suivant :

Théorème 3 Il existe des fonctions $f(x, y, t) \in C^\infty$ telles que l'équation $A(u) = f$ n'admette aucune solution $C^{1,\alpha}$ en aucun point de $\mathbb{R}^3$.

Pour plus de précisions sur cet exemple, on pourra consulter [2] ou [3]. La méthode PINN pourrait à tort nous faire croire sur cet exemple qu'une solution existe, ce qui n'est pas le cas.

Ainsi, si les PINN constituent une alternative élégante et puissante, notamment pour des problèmes de grande dimension ou mal contraints par les données, ils ne surpassent pas les méthodes traditionnelles dans les cas simples en dimension 1. Leur intérêt apparaît surtout lorsque les méthodes classiques deviennent coûteuses ou difficiles à appliquer, tandis que leur principal frein reste le temps d'exécution. Les PINN doivent donc être envisagés comme un outil complémentaire, particulièrement pertinent dans des contextes où la flexibilité et l'absence de maillage constituent un avantage décisif.

A Codes Python

A.1 Construction des PINN

A.1.1 Réseau de neurones

```
import torch
from torch.autograd import grad
```

```
class ResNeur(nn.Module):

        def __init__(self):
            super().__init__()

            self.net = nn.Sequential(
                nn.Linear(2, 1000),
                nn.ReLU(),
                nn.Linear(1000, 1)
            )

        def forward(self, x):
            return self.net(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = ResNeur().to(device)
```

A.1.2 Fonction g

```
import torch
from torch.autograd import grad
```

```
class fonc_g(nn.Module):

        def __init__(self):
            super().__init__()

        def forward(self, x):
            return torch.zeros_like(x)

fonction_g = fonc_g().to(device)
```

A.2 Tests et fonctions perte

A.2.1 Calculs préliminaires

```
import torch
from torch.autograd import grad
```

```
def calculs_preliminaires(model, N, L, fonction_g, device=None):
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    t = torch.rand(N, 1, device=device, requires_grad=True) * float(L)

    g_vals = fonction_g(t)
    inp = torch.cat([g_vals, t], dim=1)

    f = model(inp)
    df_dt = grad(f, t, grad_outputs=torch.ones_like(f), create_graph=True)[0]
    d2f_dt2 = grad(df_dt, t, grad_outputs=torch.ones_like(df_dt), create_graph=True)[0]

    return t, g_vals, f, df_dt, d2f_dt2
```

A.2.2 Tests des calculs préliminaires

```
import torch
```

```
def test_calculs_preliminaires():
    print("TEST : calculs_preliminaires()")

    print("[TEST 1] : y(t) = 5")

    model_c = ResNeur().to(device)
    with torch.no_grad():
        model_c.net[0].weight.fill_(0)
        model_c.net[0].bias.fill_(0)
        model_c.net[2].weight.fill_(0)
        model_c.net[2].bias.fill_(0)

    def fonction_g_test1(t):
        return torch.zeros_like(t)

    t, g_vals, f, df_dt, d2f_dt2 = calculs_preliminaires(model_c, N=100, L=1.0,fonction_g=fonction_g_test1, device=device)

    print(f"Max|dy/dt|= {df_dt.abs().max().item():.2e}")
    print(f"Max|d  y/dt  |= {d2f_dt2.abs().max().item():.2e}")
    print(f"TEST 1 {'PASSÉ' if df_dt.abs().max().item() < 1e-4 else 'ÉCHOUÉ'}")

    print("[TEST 2] : y(t)     2t + 1")
```

```
23
24      model_l = ResNeur().to(device)
25
26      def fonction_g_test2(t):
27          return torch.zeros_like(t)
28
29      t_b = torch.linspace(0, 1, 500, device=device).unsqueeze(1)
30      g_b = torch.zeros_like(t_b)
31      inp_b = torch.cat([g_b, t_b], dim=1)
32      y_b = 2*t_b + 1
33      optimizer = torch.optim.Adam(model_l.parameters(), lr=0.01)
34      for epoch in range(100):
35          y_pred = model_l(inp_b)
36          perte = ((y_pred - y_b)**2).mean()
37          optimizer.zero_grad()
38          perte.backward()
39          optimizer.step()
40
41      t, g_vals, f, df_dt, d2f_dt2 = calculs_preliminaires(model_l, N=100, L
    =1.0,fonction_g=fonction_g_test2, device=device)
42      moyenne_df_dt = df_dt.mean().item()
43
44      print(f"Moyenne(dy/dt)= {moyenne_df_dt:.4f}")
45      print(f"Max|d y/dt |= {d2f_dt2.abs().max().item():.2e}")
46      print(f"TEST 2 {'PASSÉ' if abs(moyenne_df_dt - 2.0) < 0.2 else 'ÉCHOUÉ
    '}")
47
48      print("RÉSUMÉ : Les 2 tests valident la fonction calculs_preliminaires
    ")
49
50  test_calculs_preliminaires()
```

A.2.3 Fonction perte 1

```
1  import torch
2  from torch.autograd import grad
```

```
1  def fonction_perte_1(model, N, L, fonction_g, device) :
2
3      t, g_vals, f, df_dt, d2f_dt2 = calculs_preliminaires(model, N, L,
    fonction_g, device)
4
5      res = torch.abs(df_dt + f - g_vals)
6      t0 = torch.zeros(1, 1, device=device)
7      tL = torch.full((1, 1), float(L), device=device)
8      g0 = fonction_g(t0)
9      f_0 = model(torch.cat([g0, t0], dim=1))
10
11     perte = (res.pow(2).sum())*(L/N) + (torch.abs(f_0-1)**2)/(L**3)
12
13     return perte
```

A.2.4 Fonction perte 2

```
import torch
from torch.autograd import grad
```

```
def fonction_perte_2(model, N, L, fonction_g, device):
    t, g_vals, f, df_dt, d2f_dt2 = calculs_preliminaires(model, N, L,
    fonction_g, device)

    res = d2f_dt2 + f - g_vals
    t0 = torch.zeros(1, 1, device=device, requires_grad=True)
    g0 = fonction_g(t0)
    inp0 = torch.cat([g0, t0], dim=1)

    f_0 = model(inp0)
    df_0 = grad(f_0, t0, grad_outputs=torch.ones_like(f_0), create_graph=
    True)[0]

    perte = (res.pow(2).sum())*(L/N) + (f_0.pow(2))/(L**3) + ((df_0-1).pow
    (2))/(L**3)

    return perte
```

A.2.5 Fonction perte 3

```
import torch
from torch.autograd import grad
```

```
def fonction_perte_3(model, N, L, fonction_g, device):
    t = torch.rand(N, 1, device=device, requires_grad=True) * float(L)
    s = torch.rand(N, 1, device=device, requires_grad=True) * float(L)

    inp = torch.cat([t, s], dim=1)
    y = model(inp)

    dt = torch.autograd.grad(y, t, grad_outputs=torch.ones_like(y),
    create_graph=True, retain_graph=True)[0]
    ds = torch.autograd.grad(y, s, grad_outputs=torch.ones_like(y),
    create_graph=True, retain_graph=True)[0]

    dtt = torch.autograd.grad(dt, t, grad_outputs=torch.ones_like(dt),
    create_graph=True, retain_graph=True)[0]
    dss = torch.autograd.grad(ds, s, grad_outputs=torch.ones_like(ds),
    create_graph=True, retain_graph=True)[0]

    laplacien = dtt + dss
    g_vals = fonction_g(inp)

    perte_edp = ((laplacien + g_vals).pow(2).mean())

    N_bord = max(1, N // 4)
    t_bord0 = torch.zeros(N_bord, 1, device=device)
    s_bord0 = torch.rand(N_bord, 1, device=device) * float(L)
```

```
    t_bord1 = torch.ones(N_bord, 1, device=device) * float(L)
    s_bord1 = torch.rand(N_bord, 1, device=device) * float(L)
    t_bord2 = torch.rand(N_bord, 1, device=device) * float(L)
    s_bord2 = torch.zeros(N_bord, 1, device=device)
    t_bord3 = torch.rand(N_bord, 1, device=device) * float(L)
    s_bord3 = torch.ones(N_bord, 1, device=device) * float(L)

    inp_bord = torch.cat([
        torch.cat([t_bord0, s_bord0], dim=1),
        torch.cat([t_bord1, s_bord1], dim=1),
        torch.cat([t_bord2, s_bord2], dim=1),
        torch.cat([t_bord3, s_bord3], dim=1),
    ], dim=0)

    y_bord = model(inp_bord)
    perte_bord = y_bord.pow(2).mean()

    return perte_edp + perte_bord
```

A.2.6 Fonction perte 4

```
import torch
from torch.autograd import grad
```

```
def fonction_perte_4(model, N, L, fonction_g, device):
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    t = torch.rand(N, 1, device=device, requires_grad=True)
    x = torch.rand(N, 1, device=device, requires_grad=True)
    inp = torch.cat([t, x], dim=1)

    y = model(inp)
    y_t = torch.autograd.grad(y, t, grad_outputs=torch.ones_like(y), create_graph=True, retain_graph=True)[0]
    y_x = torch.autograd.grad(y, x, grad_outputs=torch.ones_like(y), create_graph=True, retain_graph=True)[0]
    y_xx = torch.autograd.grad(y_x, x, grad_outputs=torch.ones_like(y_x), create_graph=True)[0]
    res = y_t - y_xx
    perte = (res.pow(2).mean())

    t_b = torch.rand(N, 1, device=device)
    x_b0 = torch.zeros_like(t_b)
    x_b1 = torch.ones_like(t_b)

    y_b0 = model(torch.cat([t_b, x_b0], dim=1))
    y_b1 = model(torch.cat([t_b, x_b1], dim=1))
    perte_bords = y_b0.pow(2).mean() + y_b1.pow(2).mean()

    x_i = torch.rand(N, 1, device=device)
```

```
    t_i = torch.zeros_like(x_i)
    y_i = model(torch.cat([t_i, x_i], dim=1))
    y_init = torch.sin(torch.pi * x_i)
    perte_init = (y_i - y_init).pow(2).mean()

    return perte + perte_bords + perte_init
```

A.3 Traitement et visualisation

A.3.1 Entraînement modèle perte

```
import torch
from torch.autograd import grad
```

```
def entrainement_modele_perte(model, N, L, fonction_g, nb_iter, device,
    fonction_perte) :

    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    liste_perte = []

    for _ in range(nb_iter) :

        perte = fonction_perte(model, N, L, fonction_g, device)

        optimizer.zero_grad()
        perte.backward()
        optimizer.step()

        liste_perte.append(perte.item())

    return liste_perte
```

A.3.2 Traces graphiques des pertes

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.rc("figure", figsize=(15,8))
```

```
def traces_des_graphiques_des_pertes(model, N, L, fonction_g, nb_iter,
    device) :

    liste_perte_1 = entrainement_modele_perte(model, N, L, fonction_g,
    nb_iter, device, fonction_perte_1)
    liste_perte_2 = entrainement_modele_perte(model, N, L, fonction_g,
    nb_iter, device, fonction_perte_2)
    liste_perte_3 = entrainement_modele_perte(model, N, L, fonction_g,
    nb_iter, device, fonction_perte_3)
    liste_perte_4 = entrainement_modele_perte(model, N, L, fonction_g,
    nb_iter, device, fonction_perte_4)

    x1 = np.arange(len(liste_perte_1))
```

```
    x2 = np.arange(len(liste_perte_2))
    x3 = np.arange(len(liste_perte_3))
    x4 = np.arange(len(liste_perte_4))

    print("Perte à la fin de l'entraînement pour l'équation 1 = ",
    liste_perte_1[-1])
    print("Perte à la fin de l'entraînement pour l'équation 2 = ",
    liste_perte_2[-1])
    print("Perte à la fin de l'entraînement pour l'équation 3 = ",
    liste_perte_3[-1])
    print("Perte à la fin de l'entraînement pour l'équation 4 = ",
    liste_perte_4[-1])

    plt.plot(x1, liste_perte_1, label="Perte équation 1")
    plt.plot(x2, liste_perte_2, label="Perte équation 2")
    plt.plot(x3, liste_perte_3, label="Perte équation 3")
    plt.plot(x4, liste_perte_4, label="Perte équation 4")

    plt.title("Graphe des pertes pour les équations différentielles")
    plt.legend(loc='best')
    plt.yscale("log")

    plt.show()
```

A.3.3 Enregistrement des modèles entraînés

```
import torch
from torch.autograd import grad
```

```
model_1 = ResNeur().to(device)
model_2 = ResNeur().to(device)

entrainement_modele_perte(model_1, 2000, 1.0, fonction_g, 2000, device,
    fonction_perte_1);
torch.save(model_1.state_dict(), "model_1.pth")

entrainement_modele_perte(model_2, 2000, 1.0, fonction_g, 2000, device,
    fonction_perte_2);
torch.save(model_2.state_dict(), "model_2.pth")
```

A.3.4 Trace graphe solution exacte et approchée ED

```
import torch
from torch.autograd import grad
```

```
def trace_solution_exacte_et_approchee_ED(model_i, fonction, device=device
    , sol_exacte, i) :

    t_vals = np.linspace(0, 1, N)
    t_vals_torch = torch.linspace(0, 1, N, device=device)

    g_vals = fonction(t_vals_torch)
```

```
    inp = torch.cat([g_vals.unsqueeze(1), t_vals_torch.unsqueeze(1)], dim
    =1)

    with torch.no_grad():
        y_approx = model_i(inp).detach().cpu().numpy().flatten()

    y_exacte = sol_exacte(t_vals)

    plt.plot(t_vals, y_exacte, label=f"Solution exacte ED {i}")
    plt.plot(t_vals, y_approx, linestyle='--', label=f"Solution approchée
    ED {i}")

    plt.title(f"Solution exacte VS solution approchée de l'ED {i}")
    plt.xlabel("t")
    plt.ylabel("y(t)")
    plt.legend(loc='best')

    plt.show()
```

A.4 Outils de comparaison des méthodes

A.5 Méthode traditionnelle

```
import torch
from torch.autograd import grad
```

```
def solution_equation_1_euler(N, T=1.0):
    t = np.linspace(0.0, T, N+1)
    y = np.zeros(N+1)
    y[0] = 1.0
    h = T / N
    for k in range(N):
        y[k+1] = y[k] + h * (-y[k])
    return t, y

t_e, y_e = solution_equation_1_euler(1000, 1.0)

y_exacte = solution_exacte_ED_1(t_e)

print("Erreur Euler:", np.max(np.abs(y_e - y_exacte)))
```

```
def solution_equation_4(nx, nt, T=1.0):
    x = np.linspace(0, 1, nx + 1)
    t = np.linspace(0, T, nt + 1)
    dx = x[1] - x[0]
    dt = t[1] - t[0]
    alpha = dt / dx**2

    if alpha > 0.5:
        print("Schéma explicite instable ")
    U = np.zeros((nt + 1, nx + 1))
```

```
11
12      U[0, :] = np.sin(np.pi * x)
13      U[:, 0] = 0.0
14      U[:, -1] = 0.0
15
16      for n in range(nt):
17          U[n+1, 1:-1] = U[n, 1:-1] + alpha * (U[n, 2:] - 2*U[n, 1:-1] + U[n
        , :-2])
18
19      return t, x, U
20
21  def sol_exacte_ED_4(t, x):
22      return np.exp(-np.pi**2 * t[:, None]) * np.sin(np.pi * x[None, :])
23
24  t, x, U = solution_equation_4(nx=100, nt=50000, T=1.0)
25  U_ex = sol_exacte_ED_4(t, x)
26
27  err = np.max(np.abs(U - U_ex))
28  print("Erreur max :", err)
29
30  plt.plot(x, U[-1], label="approx : t=T")
31  plt.plot(x, U_ex[-1], "--", label="exacte : t=T")
32  plt.legend(loc='best')
33  plt.title("Equation de la chaleur")
34  plt.show()
```

A.6 Temps d'éxécution

```
1  start = time.time()
2  t_e, y_e = solution_equation_1_euler(999, 1.0)
3  temps_methode_traditionnelle = time.time() - start
4  y_exacte = sol_exacte_ED_1(t_e)
5  err_methode_traditionnelle = np.max(np.abs(y_e - y_exacte))
6
7  start = time.time()
8  liste_perte_1 = entrainement_modele_perte(model, 2000, 1.0, fonction_g,
       2000, device, fonction_perte_1)
9  temps_pinn = time.time() - start
10
11 t_vals = np.linspace(0, 1, 1000)
12 t_vals_torch = torch.linspace(0, 1, 1000, device=device)
13 g_vals = fonction_g(t_vals_torch)
14 inp = torch.cat([g_vals.unsqueeze(1), t_vals_torch.unsqueeze(1)], dim=1)
15 with torch.no_grad():
16     y_pinn = model(inp).detach().cpu().numpy().flatten()
17 err_pinn = np.max(np.abs(y_pinn - y_exacte))
18
19 print(f"Temps méthode traditionnelle : {temps_methode_traditionnelle:.4f}
       s, Erreur max : {err_methode_traditionnelle:.6e}")
20 print(f"Temps méthode PINN : {temps_pinn:.4f} s, Erreur max : {err_pinn:.6
       e}")
```

```python
start = time.time()
t, x, U = solution_equation_4(nx=100, nt=50000, T=1.0)
temps_methode_traditionnelle_2 = time.time() - start
U_ex = sol_exacte_ED_4(t, x)
err_methode_traditionnelle_2 = np.max(np.abs(U - U_ex))

model = ResNeur().to(device)
start = time.time()
liste_perte_2 = entrainement_modele_perte(model, 2000, 1.0, fonction_g, 2000, device, fonction_perte_4)
temps_pinn = time.time() - start

t_vals_pinn = np.ones(len(x)) * 1.0
x_vals_pinn = x
inp_pinn = torch.tensor(np.column_stack([t_vals_pinn, x_vals_pinn]), dtype=torch.float32, device=device)
with torch.no_grad():
    y_pinn = model(inp_pinn).detach().cpu().numpy().flatten()
err_pinn = np.max(np.abs(y_pinn - U_ex[-1, :]))

print(f"Temps méthode traditionnelle : {temps_methode_traditionnelle_2:.4f} s, Erreur max : {err_methode_traditionnelle_2:.6e}")
print(f"Temps méthode PINN : {temps_pinn:.4f} s, Erreur max : {err_pinn:.6e}")
```

Références

[1] Cybenko, *Approximation by superpositions of a sigmoidal function*, Springer, 1989.

[2] Smoller, *Shock waves and reaction diffusion equations*, Springer.

[3] John Fritz, *Partial Differential Equation*, 3ème édition, Springer.

[4] Andrew Barron, *Approximation and Estimation Bounds for Artificial Neural Networks* , Kluwer Academic Publishers, 1994.

[5] Lawrence Evans, *Partial differential equations*, 2nd édition, American Mathematical Society, 2010.